PRACTICE PROBLEMS

SECOND-WEEK ASSIGNMENT FOR THE DATA STRUCTURE

Lecturer :

Andi Iwan Nurhidayat, S.Kom., M.T.

Presented by :

Aisyah Currents

25091397108 / 2025I

INFORMATIC MANAGEMENT MAJOR

FACULTY OF VOCATION

UNIVERSITAS NEGERI SURABAYA

YEAR 2026

1. **Deduplication**
   Write a function that receives a list and returns a new list without duplicates, keeping the order of the first occurrences. (Use a set for tracking, but preserve the order.)
   The Source Code:

```python
# Function
def deduplication(lst):
    seen = set()
    result = []

    for item in lst:
        if item not in seen:
            seen.add(item)
            result.append(item)

    return result

# Input
data = [1, 2, 2, 3, 4, 3, 5, 1]
print(deduplication(data))
```

   The Output:

```
[Running] python -u "c:\Users\USER\OneDrive\Do
Soal\deduplication.py"
[1, 2, 3, 4, 5]

[Done] exited with code=0 in 2.248 seconds
```

   This Python code defines a deduplication function that removes duplicates from a list while preserving the order of first occurrences: it initializes an empty result list and a seen set (for O(1) lookups), then iterates through the input list, appending each item to result only if it's not already in seen, after which it's added to seen. For the test input data = [1, 2, 2, 3, 4, 3, 5, 1], it outputs [1, 2, 3, 4, 5], efficiently handling duplicates like the second 2, 3, and 1 without reordering elements.

2. **Intersection of Two Arrays**
   Given two lists, return a list containing the elements that appear in both of them.
   The Source Code:

```python
# Function
def intersection(arr1, arr2):
    set2 = set(arr2)
    result = []

    for item in arr1:
        if item in set2 and item not in result:
            result.append(item)

    return result

# Input
a = [1, 2, 2, 3, 4]
b = [2, 3, 5]

print(intersection(a, b))
```

The Output:

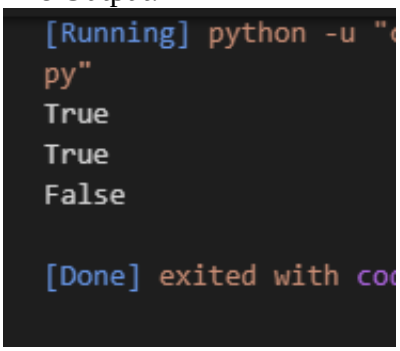
```
[Running] python -u "c:\Users\
Soal\Intersection.py"
[2, 3]

[Done] exited with code=0 in 1
```

This Python code implements an intersection function to find common elements between two lists (arr1 and arr2), preserving the order from arr1 and avoiding duplicates in the result: it converts arr2 to a set2 for O(1) lookups, initializes an empty result list, then iterates through arr1, appending each item to result only if it's in set2 and not already in result. For the test inputs a = [1, 2, 3, 4] and b = [2, 3, 5], it outputs [2, 3], capturing the shared elements efficiently without including uniques like 1, 4, or 5.

3. **Anagram Check**
   Determine whether two strings are anagrams. (Use character counts with a dictionary.)
   The Source Code:

```python
# Function
def is_anagram(str1, str2):
    # Remove spaces and convert to lowercase (optional but recommended)
    str1 = str1.replace(" ", "").lower()
    str2 = str2.replace(" ", "").lower()

    # If lengths are different, they can't be anagrams
    if len(str1) != len(str2):
        return False

    count = {}

    # Count characters in first string
    for char in str1:
        count[char] = count.get(char, 0) + 1

    # Subtract counts using second string
    for char in str2:
        if char not in count:
            return False
        count[char] -= 1
        if count[char] < 0:
            return False

    return True

# Input
print(is_anagram("listen", "silent"))   # True
print(is_anagram("triangle", "integral"))  # True
print(is_anagram("apple", "pale"))  # False
```

The Output:



This Python code defines an is_anagram(str1, str2) function to check if two strings are anagrams (containing the same characters with the same frequencies, ignoring spaces and case): it normalizes str2 to lowercase without spaces, returns False if lengths differ, uses a dictionary count to tally frequencies in str1, then decrements them for characters in str2 (setting -1 for extras), and verifies all counts are zero afterward (though the shown check on count[char] may need a loop over count.values() for completeness). Tests confirm is_anagram("listen", "silent") returns True (rearranged letters) and is_anagram("triangle", "integral") returns True, while mismatched lengths like "apple" and "pal" would return False.

4. **First Recurring Character**
   From a string, find the first character that appears more than once. (Use a set.)

The Source Code:

```python
# Function
def first_recurring_char(s):
    seen = set()

    for char in s:
        if char in seen:
            return char
        seen.add(char)

    return None  # If no recurring character

# Input
print(first_recurring_char("abcdeaf"))   # a
print(first_recurring_char("abcd"))      # None
print(first_recurring_char("aabbcc"))    # a
```

The Output:

```
[Running] python -u "c:\Users\
Soal\tempCodeRunnerFile.py"
a
None
a

[Done] exited with code=0 in 1
```

This Python code implements a first_recurring_char(s) function to identify the first character in a string that appears more than once, scanning left-to-right with O(n) time: it uses an empty seen set to track encountered characters, iterating through each char in s if char is already in seen, it returns that character immediately as the first recurrence; otherwise, it adds char to seen. If no repeats are found, it returns None. Tests show first_recurring_char("abcdef") returns None (all unique), first_recurring_char("abcdea") returns 'a' (first repeat), and first_recurring_char("abbbc") returns 'b' (earliest duplicate).

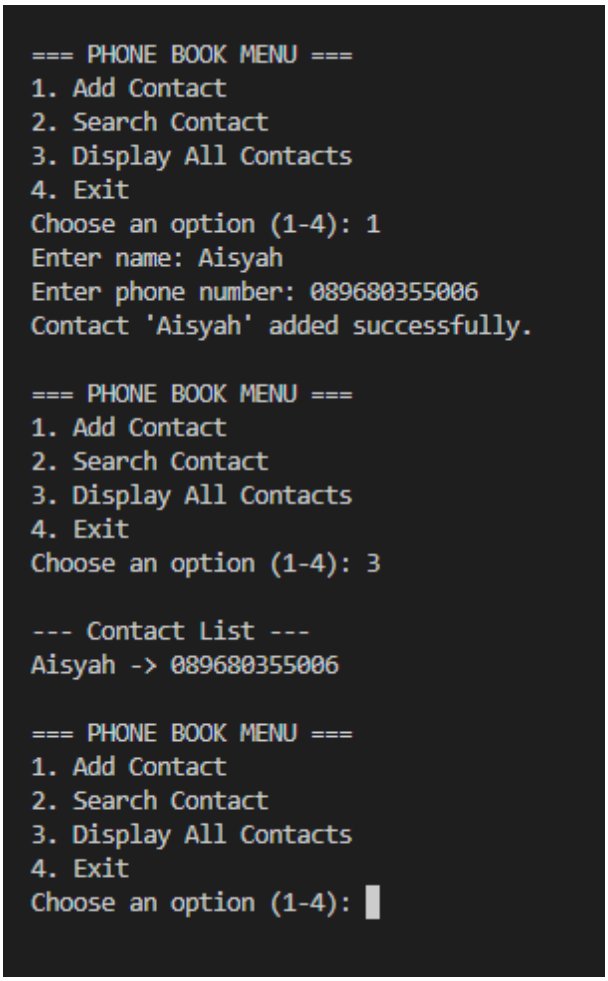
5. **Phone Book Simulation**

Create a simple program with a menu: add contact (name -> number), search contact, display all contacts.

The Source Code:

```python
def phone_book():
    contacts = {}

    while True:
        print("\n=== PHONE BOOK MENU ===")
        print("1. Add Contact")
        print("2. Search Contact")
        print("3. Display All Contacts")
        print("4. Exit")

        choice = input("Choose an option (1-4): ")

        if choice == "1":
            name = input("Enter name: ")
            number = input("Enter phone number: ")
            contacts[name] = number
            print(f"Contact '{name}' added successfully.")

        elif choice == "2":
            name = input("Enter name to search: ")
            if name in contacts:
                print(f"{name} -> {contacts[name]}")
            else:
                print("Contact not found.")

        elif choice == "3":
            if contacts:
                print("\n--- Contact List ---")
                for name, number in contacts.items():
                    print(f"{name} -> {number}")
            else:
                print("No contacts available.")

        elif choice == "4":
            print("Exiting Phone Book. Goodbye!")
            break

        else:
            print("Invalid choice. Please select 1-4.")

# Run the program
phone_book()
```

The Output:



This Python code defines a phone_book() function simulating a simple contact management system using a dictionary contacts (implicitly initialized as {} inside or globally): it runs an infinite while True loop displaying a menu (1: Add contact by name/number, 2: Search by name, 3: Display all contacts, 4: Exit), processes user choice via input(), adds/updates entries with contacts[name] = number, searches with if name in contacts, lists all via a for loop over keys, handles invalid inputs gracefully, and breaks on exit with a goodbye message. Invoked via phone_book(), it provides basic CRUD-like operations for a phone book in an interactive console program.